Quality Inspection Tracker

A mobile-first full-stack Quality Inspection Tracker built to digitize shop-floor quality defect management in manufacturing environments. The application enables supervisors to log inspections, classify defects by type and severity, track and filter open/resolved issues, and manage resolution workflows through a responsive web interface. Built with Angular, Java, Spring Boot, RESTful APIs, JPA/Hibernate, SQLite, and Docker/Docker Compose.

Quick Start (< 5 minutes)

Prerequisites

- Java 21 (JDK)
- Node.js 18+ & npm
- Maven 3.8+
- Docker Desktop v4.81.0

Option 1: Run with Docker (Recommended)

Make sure **Docker Desktop** is installed and running on your system.

From the project root directory, run:

```
docker-compose up --build
```

This command builds the required Docker images and starts the application services.

Once the containers are running, access the application at:

http://localhost:4200

Option 2: Run Manually

Backend (Terminal 1):

```
cd backend
./mvnw spring-boot:run
```

Or on Windows:

```
cd backend
mvnw.cmd spring-boot:run
```

Frontend (Terminal 2):

```
cd frontend
npm install
npm start
```

Access the app at http://localhost:4200

API Endpoints

Method	Endpoint	Description
POST	`/api/inspections`	Create new inspection
GET	`/api/inspections`	Get all inspections (with filters)
GET	`/api/inspections/{id}`	Get inspection by ID
PATCH	`/api/inspections/{id}/resolve`	Resolve inspection
GET	`/api/inspections/summary`	Get summary statistics
POST	`/api/sap-webhook`	SAP integration webhook

Filter Parameters (GET /api/inspections)

- `severity`: CRITICAL, MAJOR, MINOR
- `status`: OPEN, RESOLVED
- `startDate`: YYYY-MM-DD
- `endDate`: YYYY-MM-DD
- `sortBy`: createdAt, inspectionDate, severity
- `sortDirection`: asc, desc

SAP Webhook Payload (POST /api/sap-webhook)

```
{
  "inspectionDate": "2024-01-15",
  "machineLineId": "LINE-001",
  "defectType": "WEAVE_DEFECT",
  "severity": "CRITICAL",
  "remarks": "Optional remarks",
  "sapReferenceId": "SAP-12345"
}
```

Defect Types: WEAVE_DEFECT, SHADE_VARIATION, HOLE_TEAR, COUNT_DEVIATION, OTHER
Severities: CRITICAL, MAJOR, MINOR

Architecture Decisions

Backend (Java 21 + Spring Boot 3.3)

- **SQLite for storage**: Chosen for zero-config setup and portability. Single file database that works immediately without installation. For production, easily swappable to PostgreSQL by changing datasource config.
- **JPA Specification pattern**: Enables clean, composable filter logic for the inspection list. Avoids writing multiple repository methods for different filter combinations.
- **DTO pattern**: Separates API contracts from database entities, allowing independent evolution and better API documentation.

Frontend (Angular 18)

- **Standalone components**: Angular 18's recommended approach, reducing boilerplate and improving tree-shaking.
- **Mobile-first CSS**: All styles target 390px width first, then scale up. Touch-friendly 44px minimum tap targets throughout.
- **No UI framework dependency**: Plain CSS variables provide theming and consistent design without adding bundle size.

General

- **Consistent API response envelope**: Every response includes `{success, message, data, timestamp}` for predictable error handling.
- **Optimistic UI updates**: Toast notifications provide immediate feedback while API calls complete.

What I Would Do Differently With More Time

1. **Offline Support (PWA)**: Implement service worker with IndexedDB for offline inspection logging, syncing when connectivity returns.
2. **Authentication**: Add JWT-based auth with role-based access control (supervisor vs. manager views).
3. **Pagination**: Current implementation loads all records. Would add cursor-based pagination for scalability.
4. **Unit & E2E Tests**: Add comprehensive test coverage with Jasmine/Karma for frontend and JUnit for backend.
5. **Real-time Updates**: WebSocket integration to show live updates when another supervisor logs an inspection.
6. **Image Attachments**: Allow photo uploads for defects using device camera.
7. **Export Functionality**: CSV/Excel export of filtered inspection data for reporting.
8. **Internationalization**: i18n support for Gujarati/Hindi interface options.

Project Structure

```
├── backend/                    # Spring Boot API
│   ├── src/main/java/com/shopfloorquality/qualityinspection/
│   │   ├── config/            # CORS configuration
```

```
│   │   ├── controller/        # REST controllers
│   │   ├── dto/               # Request/Response DTOs
│   │   ├── entity/            # JPA entities
│   │   ├── exception/         # Global exception handling
│   │   ├── repository/        # Data access layer
│   │   └── service/           # Business logic
│   └── src/main/resources/
│       └── application.properties
│
├── frontend/                   # Angular 18 SPA
│   └── src/
│       ├── app/
│       │   ├── components/    # UI components
│       │   ├── models/        # TypeScript interfaces
│       │   └── services/      # API services
│       ├── index.html
│       └── styles.css         # Global mobile-first styles
│
├── docker-compose.yml
└── README.md
```

Tech Stack

- **Backend**: Java 21, Spring Boot 3.3, Spring Data JPA, SQLite
- **Frontend**: Angular 18, TypeScript 5.4, RxJS
- **Database**: SQLite (file-based, zero config)
- **Styling**: Custom CSS with CSS Variables (no framework)